Numerical Experiments

Polynomial Interpolation Over Unit Triangle

Florian Grifone

Supervisors: Pr. Stefan A. Sauter, Chuhe Lin

6. December 2023

Contents

1 Goal of the experiment **1**

2 Recap Polynomial Interpolation on $\mathbb{R}$ **2**

3 Equidistant nodes Lagrange interpolation on Unit Triangle **5**
3.1 Theory 5
3.2 Results 6

4 Gauss Lobatto nodes Lagrange Interpolation on Unit Triangle **10**
4.1 Theory 10
4.2 Results 14

5 Conclusion **i**

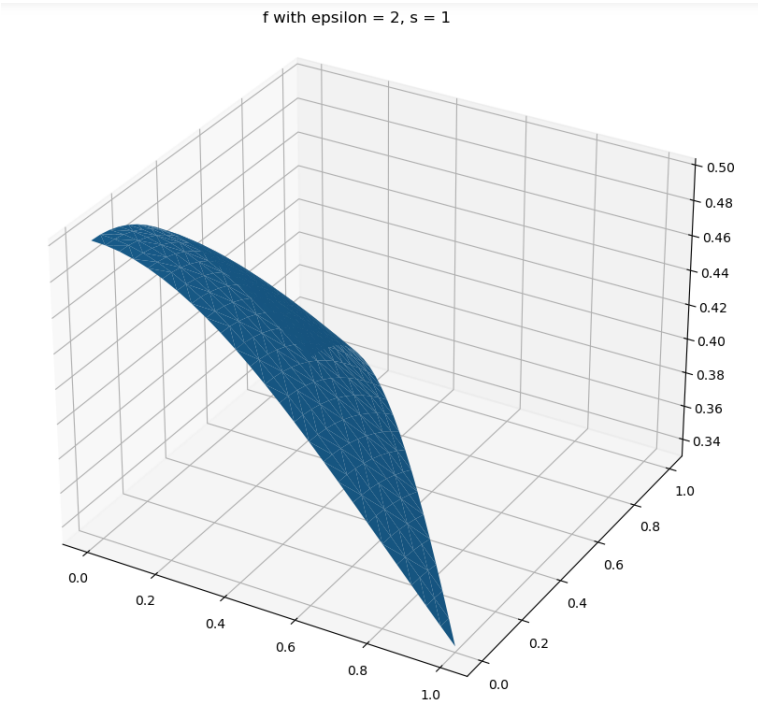
1 Goal of the experiment

Interpolation is a field of study in Numerical Analysis, as a mean of approximating functions in finitely many points. While one dimensional interpolation on the real line is well understood, extending the theory onto higher dimensions is challenging and expectedly not all results from one dimension carry over.
In this experiment we limit ourselves to two dimensions, and try to use polynomial interpolation on the function:

$$f_{\epsilon,s} : \mathcal{T} \longrightarrow \mathbb{R} \tag{1}$$

$$f_{\epsilon,s}(x,y) = \frac{1}{(x^2+y^2+\epsilon)^s} \tag{2}$$

With $\mathcal{T} = \{(x,y) \in \mathbb{R}^2 \text{ s.t. } 0 \leq x, y \leq 1, x+y \leq 1\}$, $s, \epsilon \in \mathbb{R}_{>0}$ and $s \in \mathbb{R} \setminus \{0\}$
This function is a very nice choice, due to the fact that it is a $\mathcal{C}^\infty(\mathcal{T}, \mathbb{R})$ class function but depending on the choice of paramters ϵ and s it can behave very nicely, be a nice function but with singular derivatives or be an almost singular function itself.
Below are two plots for different ϵ and s:

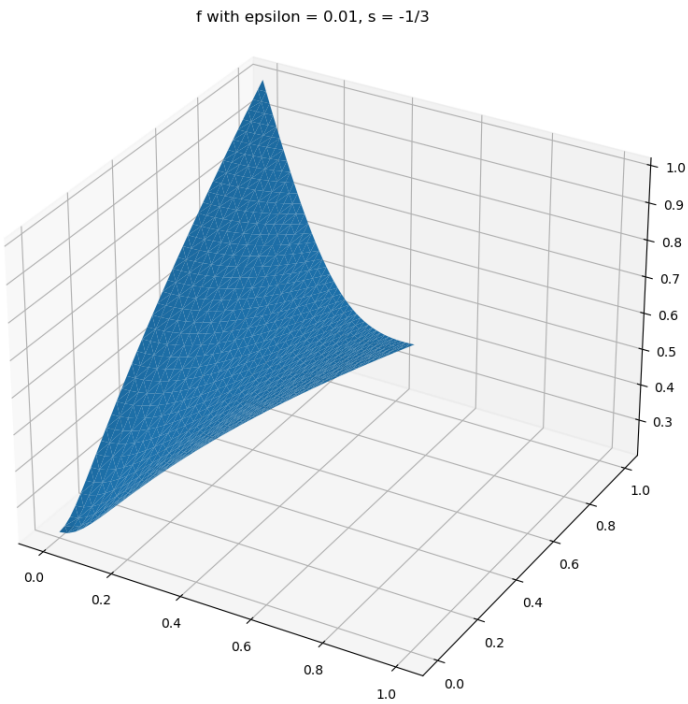


The first one is the function $f_{2,1}(x,y) = \frac{1}{x^2+y^2+2}$. It has a maximum at the origin but it is rather flat ($\frac{1}{3} \leq f_{2,1} \leq \frac{1}{2}$), and thus its derivatives behave nicely aswell.

Compared to this, the second one $f_{0.01,-1/3} = (x^2+y^2+0.01)^{\frac{1}{3}}$ has a minimumm at the origin, and its derivatives recieve a singularity at the origin.

In the first part of this experiment, we implemented interpolation with Lagrange polynomials in equidistant nodes. We expect this interpolation to work well on "nice" functions such as $f_{2,1}$, however it should struggle as functions become singular. Additionally due to Lagrange polynomials being very oscillatory, we also expect the function to have increased error when using too high polynomial degree.

In the second part we used a more optimized distribution of nodes and created a Lagrange basis using so called Proriol polynomials, to counteract the issued that arise in equidistant interpolation.
First let us briefly refresh some theory about polynomial interpolation in $\mathbb{R}$



2 Recap Polynomial Interpolation on $\mathbb{R}$

Suppose that $f \in \mathcal{C}([a,b],\mathbb{R})$ with $-\infty < a < b < \infty$ is a continuous function and we define $\Theta_n = \{x_0, x_1, ..., x_n\} \subset [a,b]$, the set of *nodes*. We say that a function u of a function class $\mathcal{S}$ interpolates f in Θ_n if and only if:

$$(\mathbb{I})\forall i = 0, ..., n : u(x_i) = f(x_i)$$

$\mathcal{S}$ is chosen to be such that all functions in $\mathcal{S}$ can be evaluated with finitely many operations (so that a computer can evaluate it). A common choice for $\mathcal{S}$ are the polynomials

$$\mathbb{P} = \{a_0 + a_1 x + ... + a_n x^n,\ n \in \mathbb{N}, a_0, ..., a_n \in \mathbb{R}\}$$

since besides being easy to evaluate, we know by the Weierstrass-theorem that they are dense in $\mathcal{C}([a,b],\mathbb{R})$. Moreover up to a fixed degree n, $\mathbb{P}_n = \{a_0 + a_1 x + ... + a_n x^n,\ a_0, ..., a_n \in \mathbb{R}\}$ is a $n+1$ dimensional subspace, thus it admits a basis. This makes it possible to linearize many problems, which in turn makes computation very fast for computers.
We recall the following result about polynomial interpolation:

Theorem 2.1. *Let $f \in \mathcal{C}([a,b],\mathbb{R})$ and $\Theta_n = \{x_0, x_1, ..., x_n\} \subset [a,b]$ be a set of nodes. Then there exists a unique $p_n \in \mathbb{P}_n$ that interpolates f in Θ_n*

Next we recall the concept of a Lagrange basis:

Definition 2.2. Let $\mathcal{L}_n = \{l_i\}_{i=0}^n$ be a basis of $\mathbb{P}_n$. We say that $\mathcal{L}_n$ is a *Lagrange basis* of $\mathbb{P}_n$ if and only if:

$$\forall i = 0, 1, ..., n : l_i(x_j) = \delta_{ij}$$

The Lagrange basis makes the construction and computation of the interpolating polynomial p_n very fast, because in this basis one can write p_n that solves $(\mathbb{I})$ as $p_n(x) = \sum_{j=0}^n f(x_j) l_j(x)$. Then for all $x_i \in \Theta_n$ one gets:

$$p_n(x_i) = \sum_{j=0}^n f(x_j) l_j(x_i) = \sum_{j=0}^n f(x_j)\delta_{ij} = f(x_i)$$

In one dimension the Lagrange basis is given by the Lagrange polynomials:

$$l_i(x) = \prod_{j=0, j\neq i}^{n} \frac{(x - x_j)}{(x_i - x_j)}$$

We have the following bound for the maximal error of the interpolation, under the condition that f has enough regularity. Recall that $||f||_\infty = \max_{x\in[a,b]} |f(x)|$ is a well defined norm on $\mathcal{C}([a,b],\mathbb{R})$.

Theorem 2.3. *Let $f \in \mathcal{C}^{n+1}([a,b],\mathbb{R})$ and suppose that $p_n \in \mathbb{P}_n$ interpolates f in $\Theta_n = \{x_0, x_1, ..., x_n\} \subset [a,b]$. We further define:*

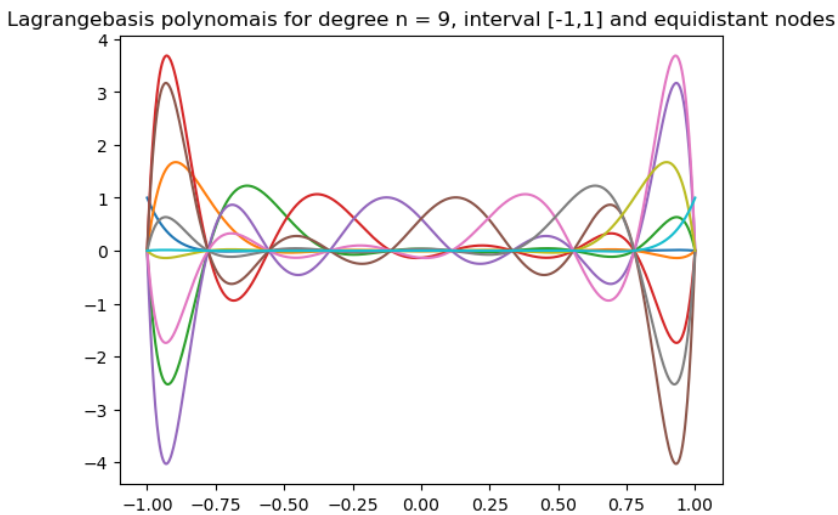
$$\omega_n(x) := (x - x_0)(x - x_1)...(x - x_n) = \prod_{j=0}^{n}(x - x_j)$$

Then:

$$||f - p_n||_\infty \leq \frac{C_{f,n}}{(n+1)!} M(\Theta_n)$$

Where $C_{f,n} = ||f^{(n+1)}||_\infty$ and $M(\Theta_n) = ||\omega_n||_\infty$,

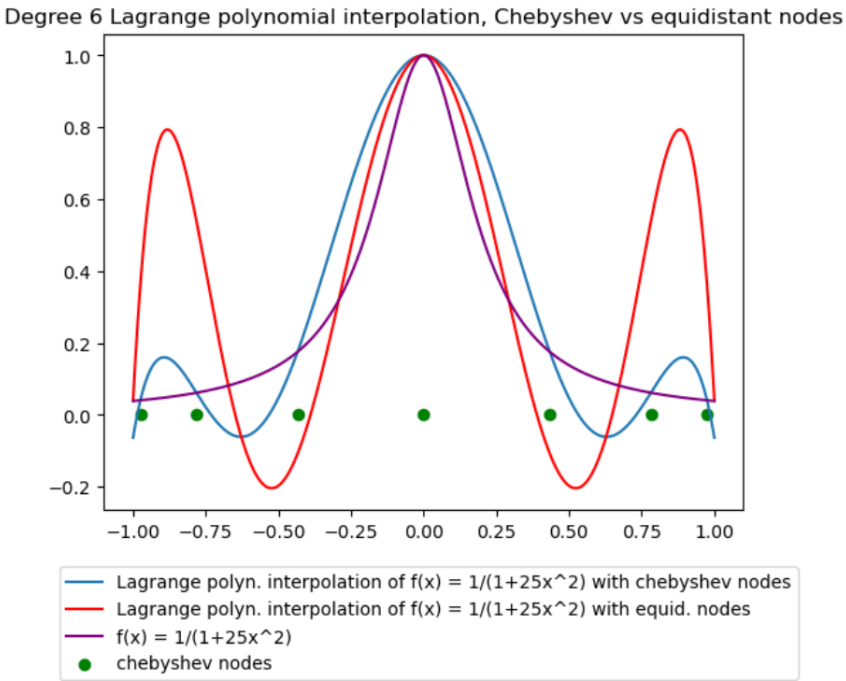
We remark that $C_{f,n}$ only depends on the function and its derivatives. Note that for example the function $f(x) = \sqrt{x+\epsilon}$ for small ϵ might be a infintely often differentiable in $[0,1]$, however the quantity $C_{f,n}$ in the error bound diverges for increasing polynomial degree n.
On the other hand $M(\Theta_n)$ depends only on the choice of nodes, hence one can try to lower the error bound with a better choice of nodes. An application of this is given by the *Runge phenomenon*. Recall that the Lagrange polynomials given in 2.2 become very oscillatory at the endpoints a, b of the interval for equidistant nodes:



This results in high errors for functions that are flat at the interval endpoints, as Carl Runge showed with the function

$$f : [-1,1] \longrightarrow \mathbb{R}$$
$$f(x) = \frac{1}{1 + 25x^2}$$

One can work against this by using the Chebyshev nodes, that are more densely distributed towards the endpoints of the intervals. Below one can see the comparison between the interpolation of the above defined function f at equidistant nodes versus Chebyshev nodes using the Lagrange polynomials of degree 6:



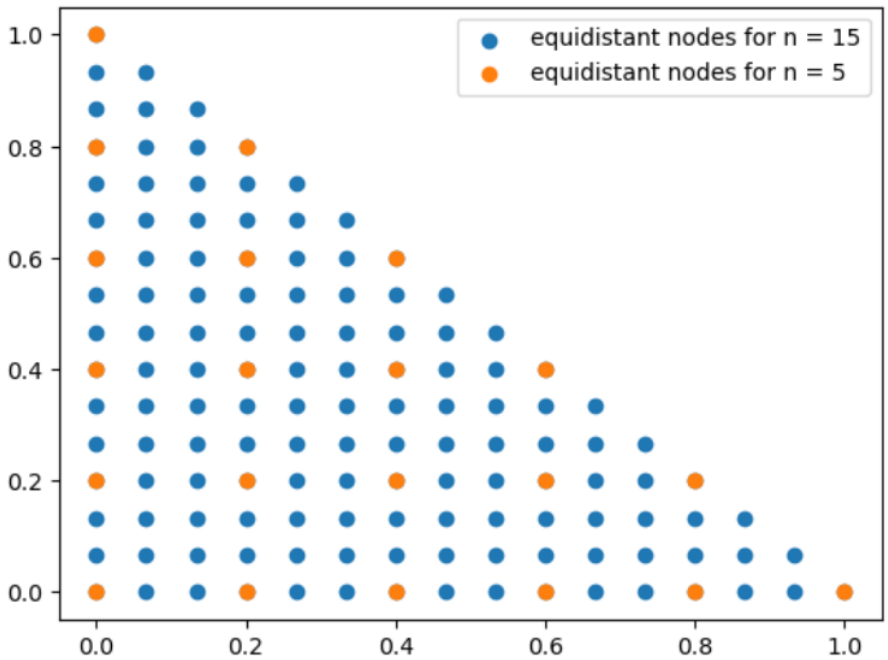
We note the significant difference performance in the interpolations at the interval endpoints, while they perform similarly at the center of the interval. We will now see many of the above discussed concepts arise again in two dimensions.

3 Equidistant nodes Lagrange interpolation on Unit Triangle

3.1 Theory

We define $\mathbb{P}_n^2 = \{\sum_{i+j\leq n}^{n} a_{ij}x^iy^j,\ a_{ij} \in \mathbb{R},\ i,j \in \mathbb{N}\}$. One can verify through induction that for each n there are $\frac{(n+1)(n+2)}{2}$ possible tuples (i,j) s.t. $i+j \leq n$.
Now suppose we create a set of nodes on the unit triangle, spaced horizontally and vertically by $\frac{1}{n}$. Formally the set is (visualized below):

$$\Theta_n^2 = \{(\frac{i}{n}, \frac{j}{n}) \in \mathbb{R}^2,\ i+j \leq n\}$$



The total amount of nodes is:

$$(n+1) + (n+1-1) + ... + 1 = \sum_{k=1}^{n+1} k = \frac{(n+1)(n+2)}{2}$$

Hence we can use a $p_n \in \mathbb{P}_n^2$ to interpolate a function $f : \mathcal{T} \longrightarrow \mathbb{R}$. The Lagrange polynomials for the equidistant nodes on $\mathcal{T}$ analogue to 2.2 are given by:

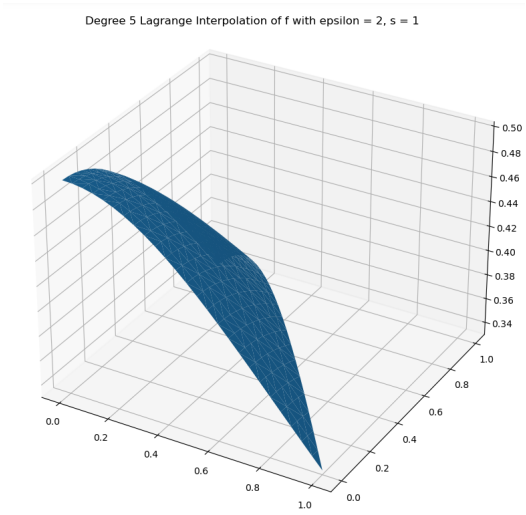
$$L_{ij}(x,y) = \frac{\prod_{l=0}^{i-1}(x-\frac{l}{n})\prod_{r=0}^{j-1}(y-\frac{r}{n})\prod_{p=0}^{k-1}((1-x-y)-\frac{p}{n})}{\prod_{l=0}^{i-1}(\frac{i}{n}-\frac{l}{n})\prod_{r=0}^{j-1}(\frac{j}{n}-\frac{r}{n})\prod_{p=0}^{k-1}((1-\frac{i}{n}-\frac{j}{n})-\frac{p}{n})}$$

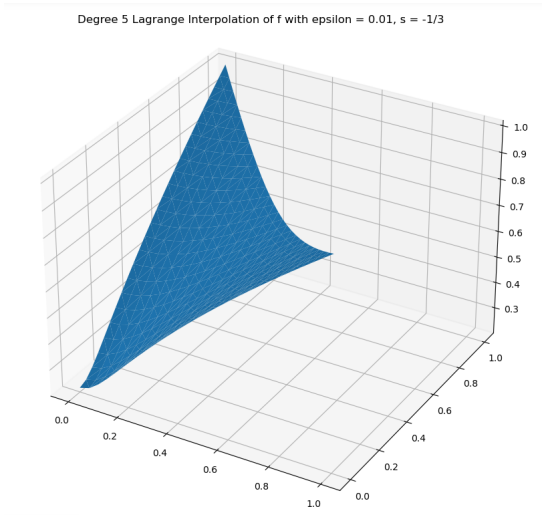
Its easy to see that $L_{ij}(\frac{i}{n}, \frac{j}{n}) = 1$ and that it is a polynomial of degree n. Through computation one could verify that $L_{ij}(\frac{\tilde{i}}{n}, \frac{\tilde{j}}{n}) = 0$, $\forall(\tilde{i}, \tilde{j}) \neq (i,j)$. The Lagrange interpolation polynomial p_n of a function $f \in \mathcal{C}(\mathcal{T}, \mathbb{R})$ is then given by:

$$p_n(x,y) = \sum_{i+j\leq n}^{n} f(\frac{i}{n}, \frac{j}{n})L_{ij}(x,y)$$

3.2 Results

We will now state the results of the experiments using the equidistant nodes Lagrange interpolation on $\mathcal{T}$ for the function $f_{\epsilon,s}(x,y) = \frac{1}{(x^2+y^2+\epsilon)^s}$. If $f_{\epsilon,s}$ was interpolated in $\frac{(n+1)(n+2)}{2}$ equidistant nodes using polynomials of degree n, then the function and the interpolation was evaluated/plotted in $\frac{(5n+1)(5n+2)}{2}$ equidistant points (Θ^2_{5n}). As a sanity, check we will plot the interpolation of degree 5 for the functions $f_{2,1}$ and $f_{0.01,-1/3}$: Visually they seem to coincide very well with the plots of the "true functions" on page 1 and 2, hence





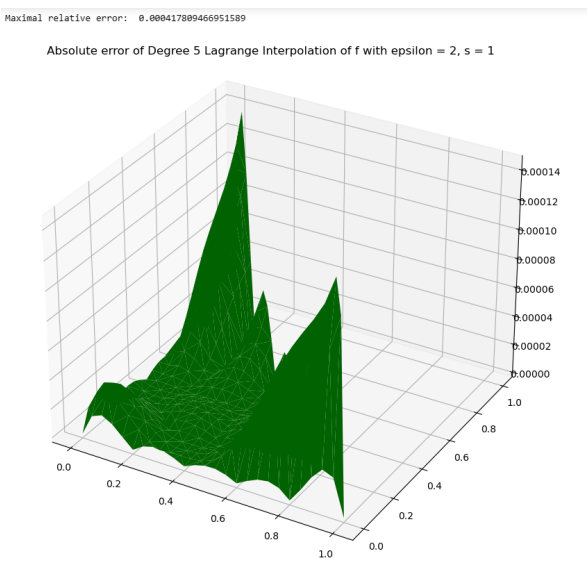
the interpolation as described in section 3.1 seems to have been implemented correctly. How did these two specific interpolations perform? For this let us look at the error plots, given by plotting the absolute difference

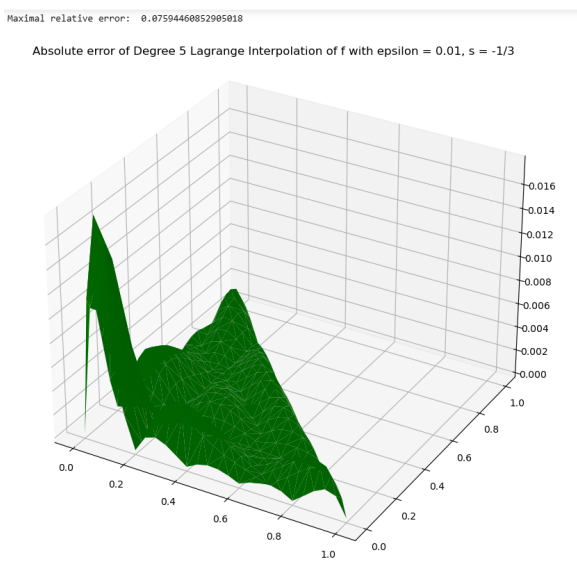
$$|f_{\epsilon,s}(x,y) - p_5(x,y)|$$

for every (x,y) in Θ^2_{25}. Additionally the maximal relative error

$$r := \max_{(x,y)\in\Theta^2_{25}} \left|\frac{f_{\epsilon,s}(x,y) - p_5(x,y)}{f_{\epsilon,s}(x,y)}\right|$$

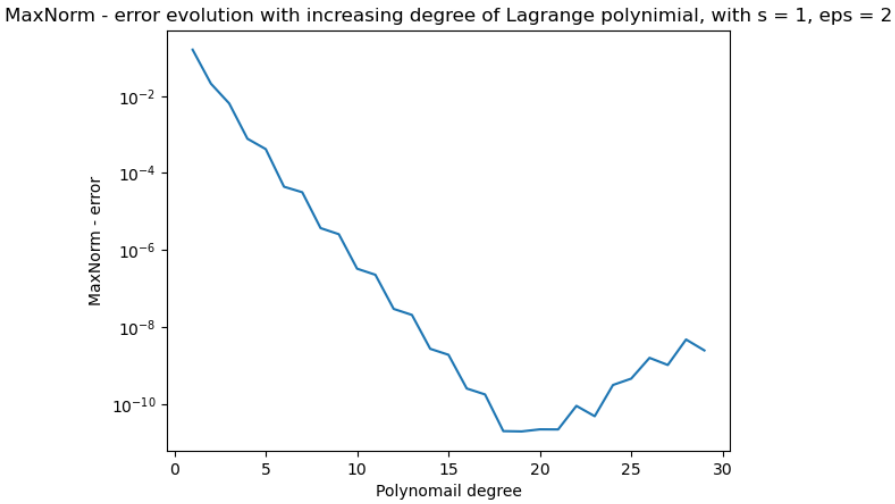
was printed for comparison. For both the function, there is very small error at the center of the triangle,

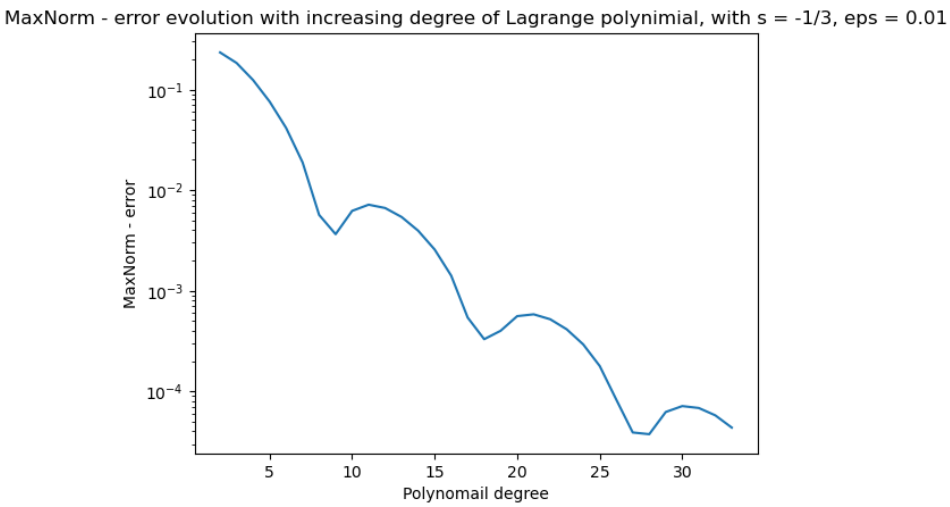




slight error on the edes and the biggest errors are at the vertices of the triangle. This coincides what we know

from one dimension: If we would be interpolating only over the edges of the triangle, it would be the same as interpolating in one dimension, and hence the biggest errors of the equidistant Lagrange interpolation also occurs on the interval endpoints. We also see that these errors in both cases look like small hills, which are due to the oscillatory nature of the Lagrange polynomials.
However the qualitative performance is very different for both: For $f_{2,1}$ the maximal relative error is $r = 0.04\%$, while for $f_{0.01,-1/3}$ we have $r = 7.5\%$ so a maximal error r that is 100-times worse. Looking at the plot the issue for $f_{0.01,-1/3}$ seems to be at the origin. A reason for this can be the fact that the function is almost zero at the origin, which is again a vertex of the triangle and hence if we think back to the Runge phenomenon from section 2 a worse performance could be explained again due to the highly oscillatory behaviour of Lagrange functions at boundaries of the regons.
On the other hand we also remark that high (partial) derivatives of $f_{0.01,-1/3}$ become more and more singular. Thus we can also use the error bound from Theorem 2.3 as an explanation for the large error compared to the interpolation of $f_{2,1}$, which does not have this issue at the origin. So for the equidistant Lagrange interpolation on the 2-dimensional unit triangle, similar behaviour seems to arise as it does on 1-dimensional intervals.
Let us now see how the maximal relative error evolves for increasing polynomial degree, instead of a fixed degree 5. We will plot the relative error r scaled logarithmically on the y-axis against the polynomial degree scaled linearly on the x-axis, for both functions $f_{2,1}$ and $f_{0.01,-1/3}$:





Again we see that for $f_{2,1}$ the error behaves very nicely; it decreases very quickly from $r \approx 10^{-1}$ for degree 1 to $r \approx 10^{-11}$ for degree 17. After that r seems to increase again a bit. A reason for this could be again the high oscillations in the degrees $20 - 30$. For $f_{0.01,-1/3}$ the curve seems in general to go down, but for some degrees it rises again. However its far less steep as the other curve, since her the error goes down from $\approx 10^{-1}$ to only $\approx 10^{-5}$. The performance thus is a lot worse for the second function, despite the function itself not having any singularity.

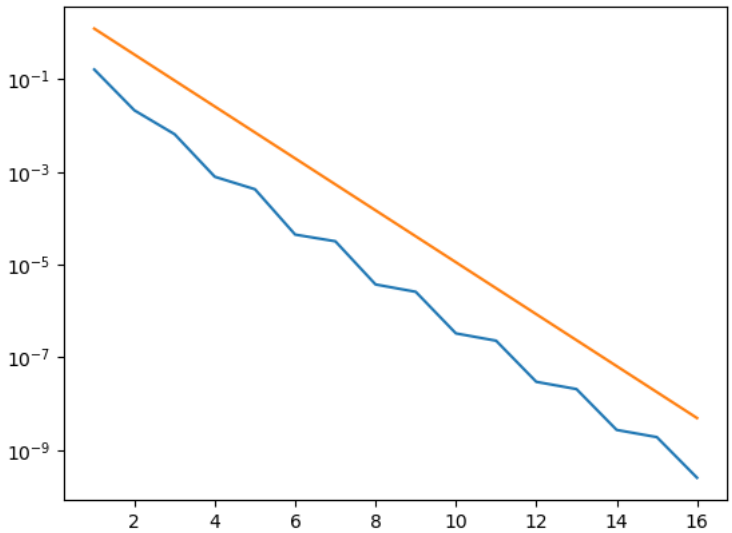
For $f_{2,1}$ between the degrees 1 and 17 we have almost linear decay in the semilog plot, thus the error can be considered to be decaying exponentially with the degree. Explicitly we have that if $r(n)$ is the maximal relative error of the interpolation with degree n, then there exist constants $C \in \mathbb{R}$ and $\alpha < 0$ s.t.

$$r(n) = Ce^{\alpha n}$$

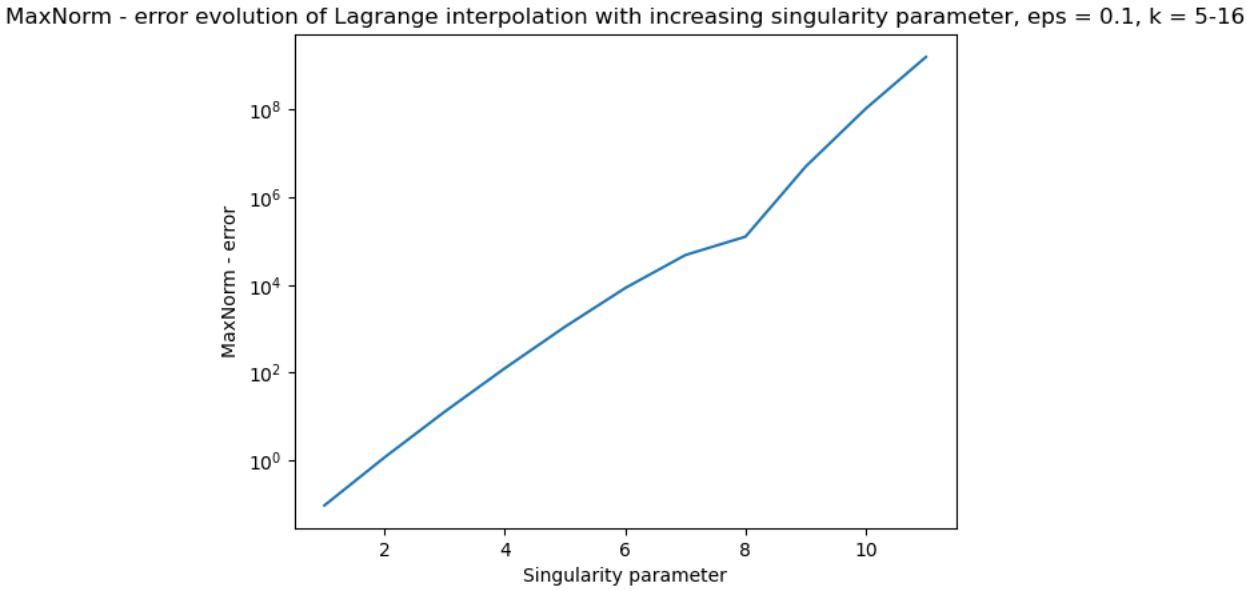
The smaller α is the steeper the curve / the faster the convergence. As an approximation of this α we computed $\log(\frac{r(n+1)}{r(n)}) = \alpha_n$ for every $n = 1, ..., 16$ and then took $\alpha = \frac{1}{16}\sum_{n=1}^{16} \alpha_n$. The result was $\alpha \approx -1.29$, below one can see the "linear" fit $r(n) = Ce^{\alpha n}$ in the semilogplot, compared with the real error decay:

```
-1.2894538979157426

[<matplotlib.lines.Line2D at 0x29ecf52d280>]
```



Finally let us test how the interpolation performs when the function becomes more and more singular. For this we fixed $\epsilon = 0.1$ and interpolated over multiple iterations in which we increased both the singularity parameter s, aswell as the polynomial degree. This way, at the origin the singularity becomes larger and larger but the interpolation has access to higher degree polynomials which should help to interpolate singular functions. The following plot shows the results, in which again the mamximal relative error for each interpolation is plotted in logarithmic scale, this time against the singularity parameter s, that ranges from 1 to 11. The polynomial degree at each iteration is equal to $s + 4$.



Not only does the error diverge exponentially, which could be expected given that for $s = 11$, the function is not really a continuous function anymore, but even when s is small for example 1 or 2, even with polynomial degree of 5 and 6, we have a maximal error of about 10% and more than 100% respectively. This further cements that Lagrange fails to interpolate functions that are even slightly singular. With this in mind we want to solve these mentioned issues that arise for the equidistant Lagrange interpolation, with a new choice of nodes.

4 Gauss Lobatto nodes Lagrange Interpolation on Unit Triangle

4.1 Theory

Similarly to the Chebyshev nodes in one dimension, which are more densely distributed at the boundaries of the region (the interval endpoints), we now want to find a set of nodes that is

- rather sparse at the center of our triangle
- dense at the edges of the triangle
- very dense at the vertices of the triangle.

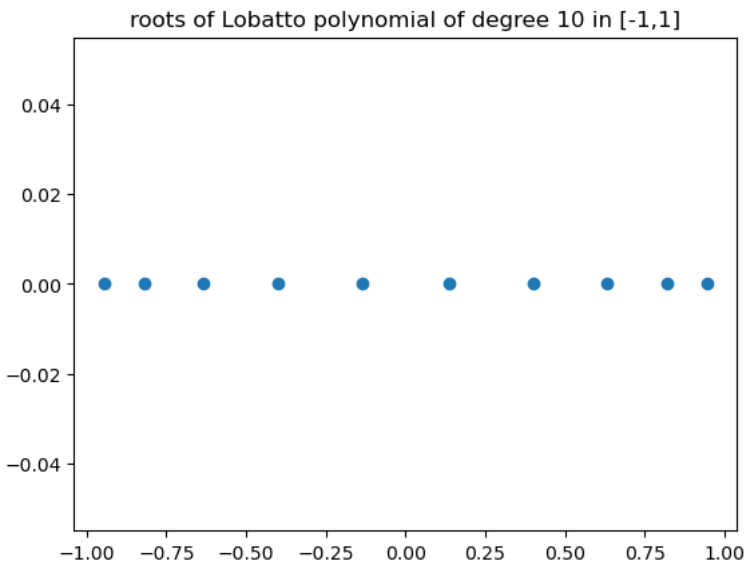
As the Chebyshev nodes are the roots of the Chebyshev polynomials, we are going to use aswell roots of special polynomials. They belong to the class of *Jacobi Polynomials*:
Let $\alpha, \beta \in \mathbb{N}$. Then the Jacobi polynomial of degree n in the interval $[-1, 1]$ is defined by:

$$P_n^{(\alpha,\beta)}(x) := \sum_{k=0}^{n} \binom{n+\alpha}{n-k} \binom{n+\beta}{k} \left(\frac{x-1}{2}\right)^k \left(\frac{x+1}{2}\right)^{n-k}$$

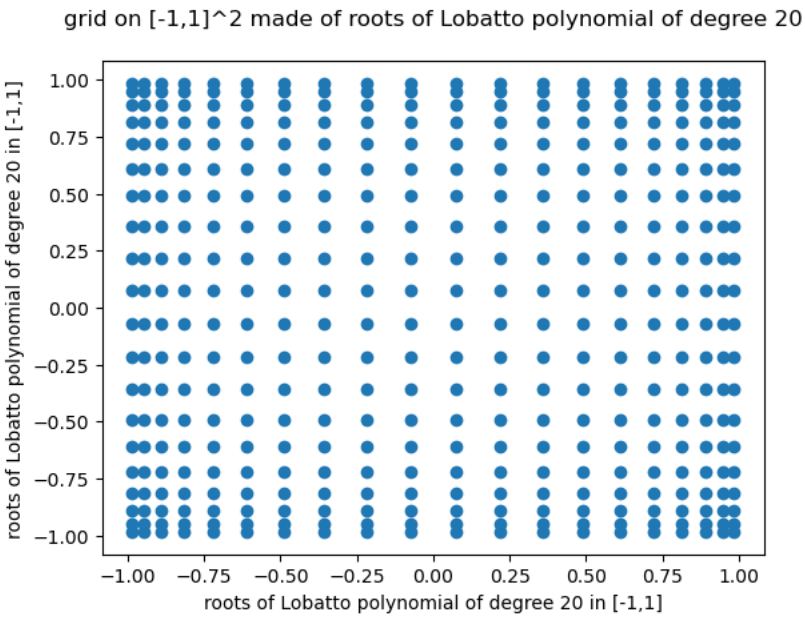
For our nodes we are going to use the roots of the *Lobatto polynomial* which is a special case of the Jacobi polynomial. For degree n on the interval $[-1, 1]$ it is given by:

$$Lo_n(x) := \frac{n+2}{2} P_n^{(1,1)}(x)$$

This is how the 10 roots of the Lobatto polynomial of degree 10 in the interval $[-1, 1]$:



Notice that they have a similar structure as the Chebyshev nodes have. Additionally they in the interior of $[-1, 1]$ and are symmetric. Now suppose we were interpolating over the unit square $[-1, 1]^2$. Then we could just take a grid constructed by having the roots of the Lobatto polynomials on the x and y-axis to get a set of nodes that has the desired distribution (sparse at center, dense at edges, very dense at vertices). Below one can see how this grid would look like for the Lobatto polynomial of degree 20:



Now we want to have nodes with this structure on $\mathcal{T}$, however on the square we have n^2 node points, while we only need $\frac{(n+1)(n+2)}{2}$ for the triangle. We will construct them in the following way;

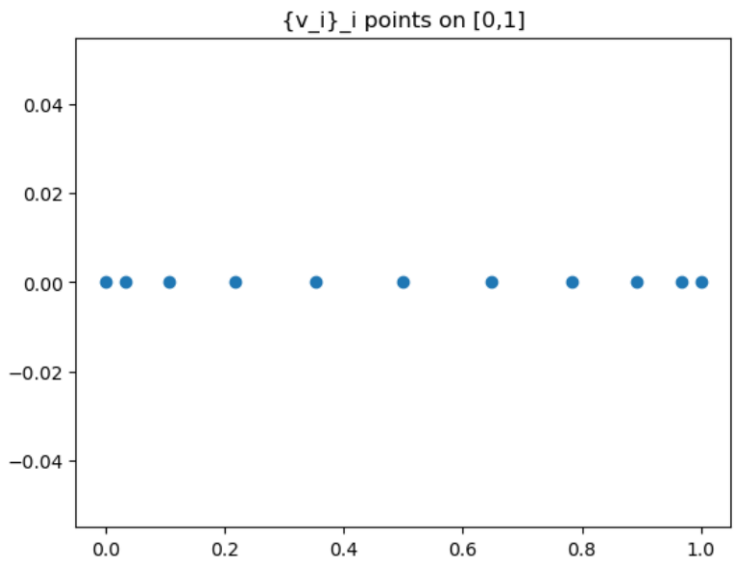
1. we define $\{t_i\}_{i=1}^{n-1} \subset (-1,1)$ to be the $n-1$ roots of the Lobatto polynomial Lo_{n-1}
2. next we define $\{v_i\}_{i=0}^{n}$ by setting $v_0 = 0$, $v_n = 1$ and for $i = 1, ..., n-1$ we set v_i to:

$$v_i = \frac{1}{2}(1 + t_i)$$

Note that since $-1 < t_i < 1$ the above for $i = 1, ..n-1$ defines a mapping to $(0,1)$. Moreover by the symmmetry of the roots we have:

$$v_i = \frac{1}{2}(1 + t_i) = \frac{1}{2}(1 - t_{n-1}) = 1 - \frac{1}{2}(1 + t_{n-1}) = 1 - v_{n-i}$$

Consequently we obtain $n+1$ points $\{v_i\}_{i=0}^{n} \subset [0,1]$ that keep the desired structure:



3. Since now we have the nodes on the intervals $[0,1]$ which form the edges on the x and y-axis of our unit triangle we need to construct the remaining nodes. We do this in the following way: for a fixed $i = 0, .., n$ we have the nodes

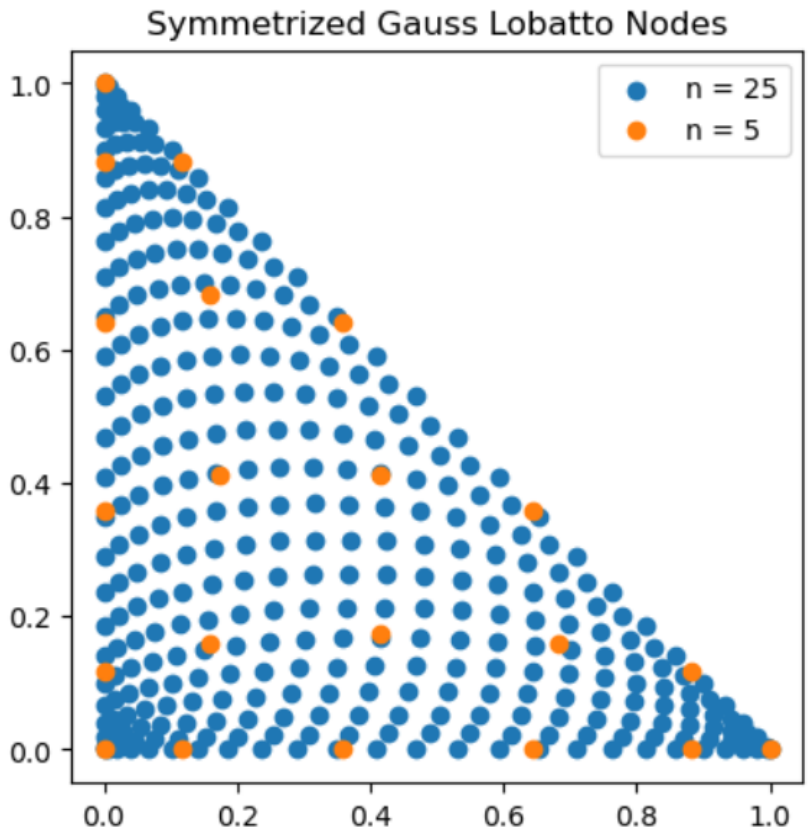
$$(x_i, y_j) = (\frac{1}{3}(1 + 2v_i - v_j - v_{n-i-j}), \frac{1}{3}(1 + 2v_j - v_i - v_{n-i-j}))$$

where $j = 0, 1, ..., n - i$. Note that if $i = 0$ we obtain the nodes:

$$\begin{aligned}(x_0, y_j) &= (\frac{1}{3}(1 + 2v_0 - v_j - v_{n-0-j}), \frac{1}{3}(1 + 2v_j - v_0 - v_{n-0-j})) \\ &= (\frac{1}{3}(1 - v_j - v_{n-j}), \frac{1}{3}(2v_j + 1 - v_{n-j})) \\ &= (\frac{1}{3}(v_{n-j} - v_{n-j}), \frac{1}{3}(2v_j + v_j)) \\ &= (0, v_j)\end{aligned}$$

Meaning we get exactly the nodes back on the interval $[0,1]$ of the y-axis. If we were to fix $j = 0$ and let i range between $0, 1, ..., n$ we would get the nodes as in the image above in the interval $[0,1]$ of the x-axis. As a total we have as many nodes, as there are combinations of of the tuple (i, j) s.t. $i + j \leq n$, which is precisely $\frac{(n+1)(n+2)}{2}$.

From this construction we obtain the *symmtetrized Gauss Lobatto nodes*:



Notice that these nodes are distributed exactly as we described them at the beginning of this section! We will denote this set as $\Theta^2_{n,GL}$, with $|\Theta^2_{n,GL}| = \frac{(n+1)(n+2)}{2}$.
Now we want to construct a Lagrange-basis for this new set of nodes. For this we will use the so called *Proriol* polynomials which are two variable polynomials that are originally defined using a Jacobi polynomial

in x in the interval $[-1,1]$ and another Jacobi polynomial in y also in the interval $[-1,1]$. Using a change of variables we can construct the Proriol polynomials for our unit triangle. For degree n with $k,l \in \mathbb{N}$ s.t. $k+l \leq n$ we have:

$$\mathcal{P}_{kl}(x,y) = P_k^{(0,0)}(x)\Big(\frac{1-y}{2}\Big)^k P_l^{(2k+1,0)}(y)$$

As an example for $n=2$ we get the $\frac{(2+1)(2+2)}{2} = 6$ polynomials of degree 2:

1. $\mathcal{P}_{00}(x,y) = 1$
2. $\mathcal{P}_{10}(x,y) = 2x+y-1$
3. $\mathcal{P}_{01}(x,y) = 3y-1$
4. $\mathcal{P}_{20}(x,y) = 6x^2+6xy+y^2-6x-2y+1$
5. $\mathcal{P}_{11}(x,y) = (2x+y-1)(5y-1)$
6. $\mathcal{P}_{02}(x,y) = 10y^2-8y+1$

While they do form a basis for $\mathbb{P}_2^2(\mathcal{T})$, they are not a Lagrange basis, as for example $\mathcal{P}_{00}$ is constantly equal to 1 and thus cant be 0 in different nodes. Thus we have to find the right change of basis matrix $B = [b_0|b_1|...|b_i|...|b_{\frac{(n+1)(n+2)}{2}}]$, $b_i \in \mathbb{R}^{\frac{(n+1)(n+2)}{2}}$ s.t. for each $(x_j,y_j) \in \Theta_{n,GL}^2$, $j = 0,1,...,\frac{(n+1)(n+2)}{2}$ we have:

$$\sum_{k+l\leq n}^{n} b_{i,kl}\mathcal{P}_{kl}(x_j,y_j) = \delta_{ij}$$

Then we can take the i-th Lagrange basis polynomial to be exactly $l_i(x,y) = \sum_{k+l\leq n}^{n} b_{i,kl}\mathcal{P}_{kl}(x,y)$. Let us write $m_n := \frac{(n+1)(n+2)}{2}$ from now on. To find these coefficients we solve the so called *Van der Monde System*, which consists for each $i = 0,1,...,m_n$ to solve:

$$\begin{bmatrix} \mathcal{P}_{00}(x_0,y_0) & \mathcal{P}_{10}(x_0,y_0) & \mathcal{P}_{01}(x_0,y_0) & \dots & \mathcal{P}_{0n}(x_0,y_0) \\ \mathcal{P}_{00}(x_1,y_1) & \mathcal{P}_{10}(x_1,y_1) & \mathcal{P}_{01}(x_1,y_1) & \dots & \mathcal{P}_{0n}(x_1,y_1) \\ \dots & \dots & \dots & \dots & \dots \\ \mathcal{P}_{00}(x_i,y_i) & \mathcal{P}_{10}(x_i,y_i) & \mathcal{P}_{01}(x_i,y_i) & \dots & \mathcal{P}_{0n}(x_i,y_i) \\ \dots & \dots & \dots & \dots & \dots \\ \mathcal{P}_{00}(x_{m_n},y_{m_n}) & \mathcal{P}_{10}(x_{m_n},y_{m_n}) & \mathcal{P}_{01}(x_{m_n},y_{m_n}) & \dots & \mathcal{P}_{0n}(x_{m_n},y_{m_n}) \end{bmatrix} \begin{bmatrix} b_{i,00} \\ b_{i,10} \\ \dots \\ b_{i,kl} \\ \dots \\ b_{i,0n} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \dots \\ 1 \\ \dots \\ 0 \end{bmatrix}$$

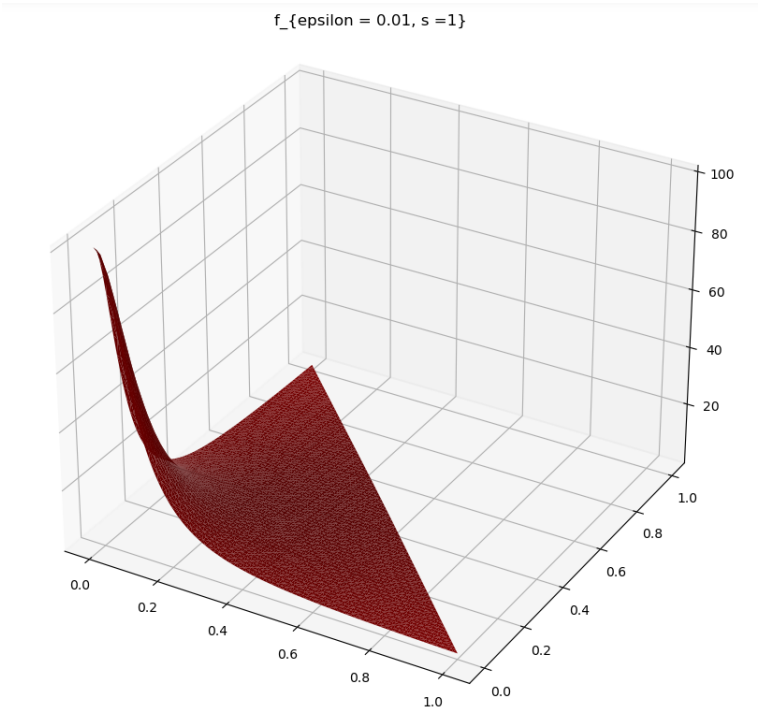
This is the most computationally expensive part of this interpolation, and thus it is worth it to use QR-docomposition on the matrix $\mathcal{P}$ once the the Gauss Lobatto nodes are fixed, as solving the linear system with a triangular matrix is much faster. Finally the interpolation polynomial for a function $f \in \mathcal{C}(\mathcal{T},\mathbb{R})$ can be then constructed as:

$$\begin{aligned} p_n(x,y) &= \sum_{i=1}^{m_n} f(x_i,y_i)l_i(x,y) \\ &= \sum_{i=1}^{m_n} f(x_i,y_i) \sum_{k+l\leq n}^{n} b_{i,kl}\mathcal{P}_{kl}(x,y) \end{aligned}$$

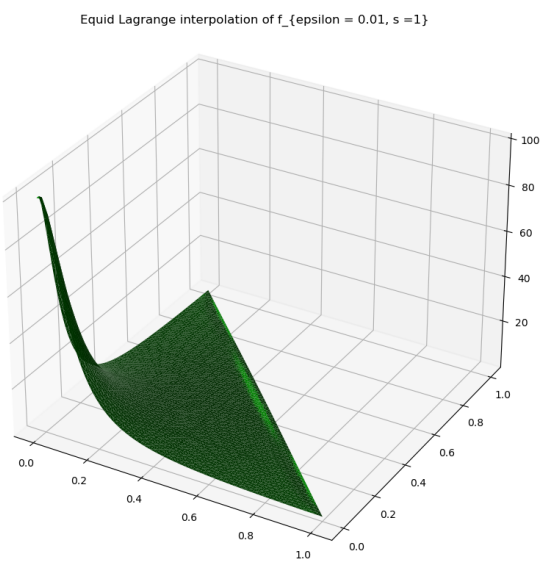
Where $(x_i,y_i) \in \Theta_{n,GL}^2$. With this implemented we can now test the results on our test function and see whether it is an improvement over the Lagrange interpolation with equidistant nodes.

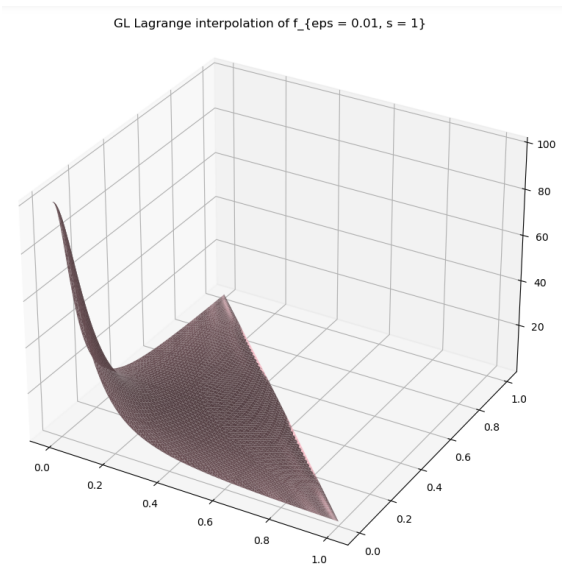
4.2 Results

As before we will first just test visually if the interpolation has been done correctly. This time we will however look at the function $f_{0.01,1}(x,y) = \frac{1}{x^2+y^2+0.1}$. We should expect the equidistant Lagrange interpolation to perform a lot worse here, as the function is very steep at the origin (a vertex of the region) and very flat elsewhere. Meanwhile we hope that the new Lagrange interpolation with the more optimal Gauss Lobatto nodes performs better. In both cases if the interpolation was done in the nodes Θ_n^2 and $\Theta_{n,GL}^2$ respectively, then the interpolations and the true functions were evaluated at the points Θ_{5n}^2 and $\Theta_{5n,GL}^2$. Below visualized is firstly the true function $f_{0.01,1}$:

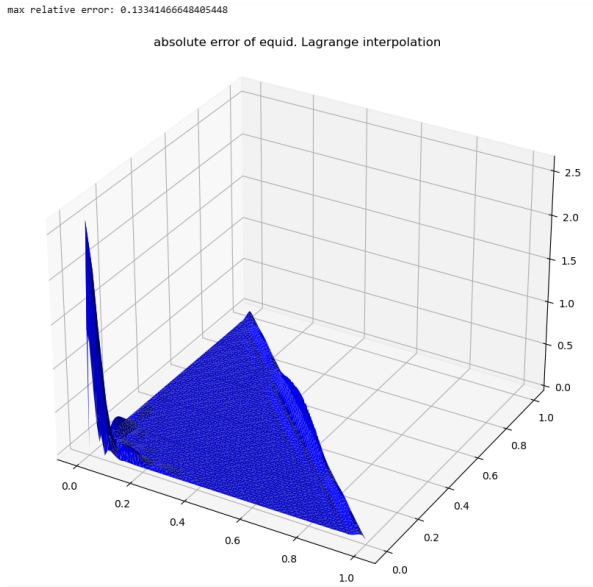


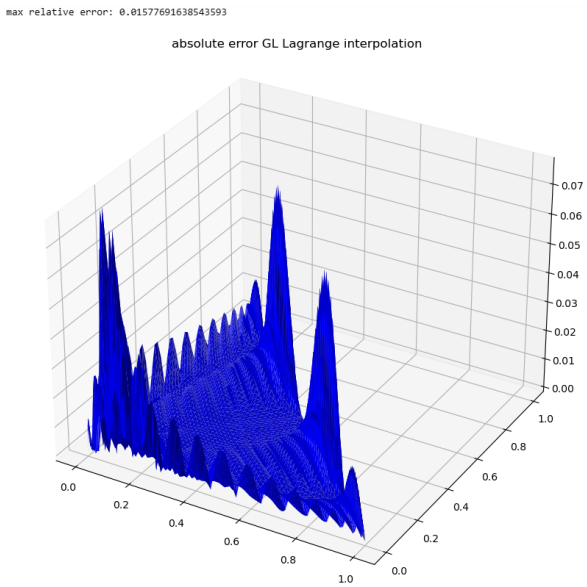
Now on the left is the Lagrange interpolation with the equidistant nodes and on the right with Gauss Lobatto nodes, both with degree 17:



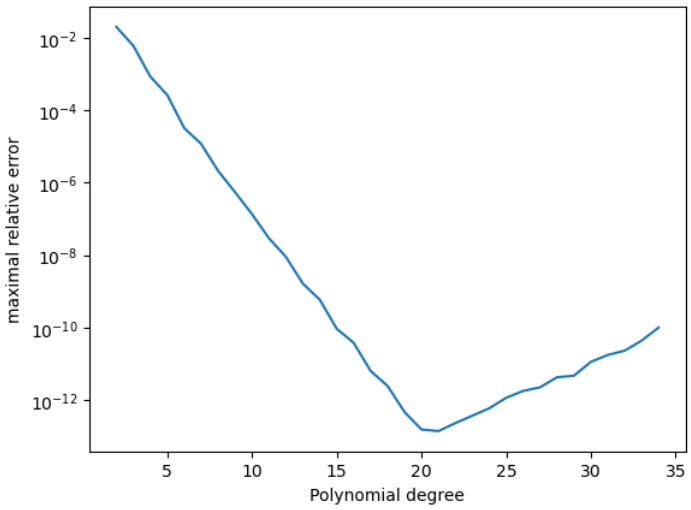


Both interpolations seem to approximate the correct function, but which one actually performs better? We will also plot here the absolute error distributions and print the maximal relative error:



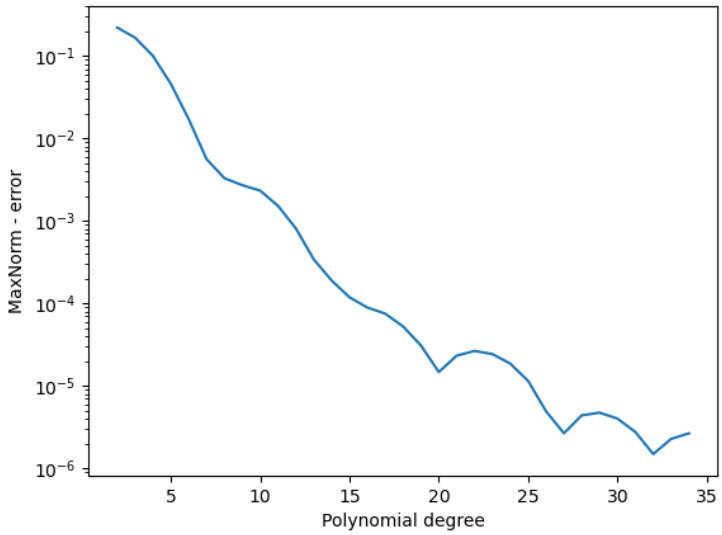


We note that while the equidistant Lagrange interpolation interpolates very well almost everywhere, it has a very large error distribution around the origin, where the singularity lies, its maximal deviation is about 13.3%. Comparatively the Gauss Lobatto Lagrange interpolation has very very small oscillatory errors at the boundary regions, similar how the Chebyshev Lagrange has in one dimensions. Its largest deviation is around 1.6%, almost 100 times better than with the equidistant nodes!
Next let us test whether it has a nice convergence for the functions we have seen before $f_{2,1}$ and $f_{0.01,-1/3}$. Here are again the the maximal relative errors scaled logarithmically against the increasing polynomial degree of the interpolation:



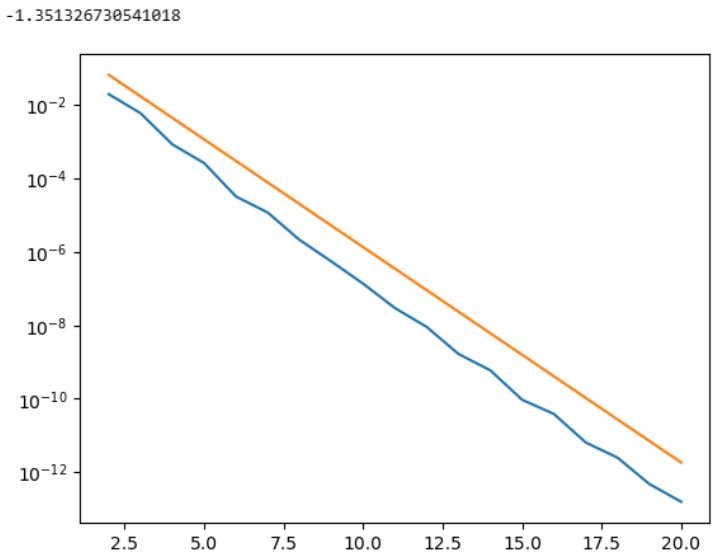
Maximal relative error evolution with GL Lagrange interpolation with $\epsilon = 2, s = 1$

For $f_{2,1}$ we have a very similar plot as with the equidistant Lagrange interpolation, the error decreases exponentailly for the first 21 degrees, then it increases again slightly. The difference to the equidistant interpolation is that the error smallest error seems around 100 times better than before ($\approx 10^{-13}$ compared to before $\approx 10^{-11}$). The interpolation also seems to be a little bit more stable for higher degree polynomials, as in the equidistant case the error increased again already for degree 17.



Maximal relative error evolution with GL Lagrange interpolation with $\epsilon = 0.01, s = -1/3$

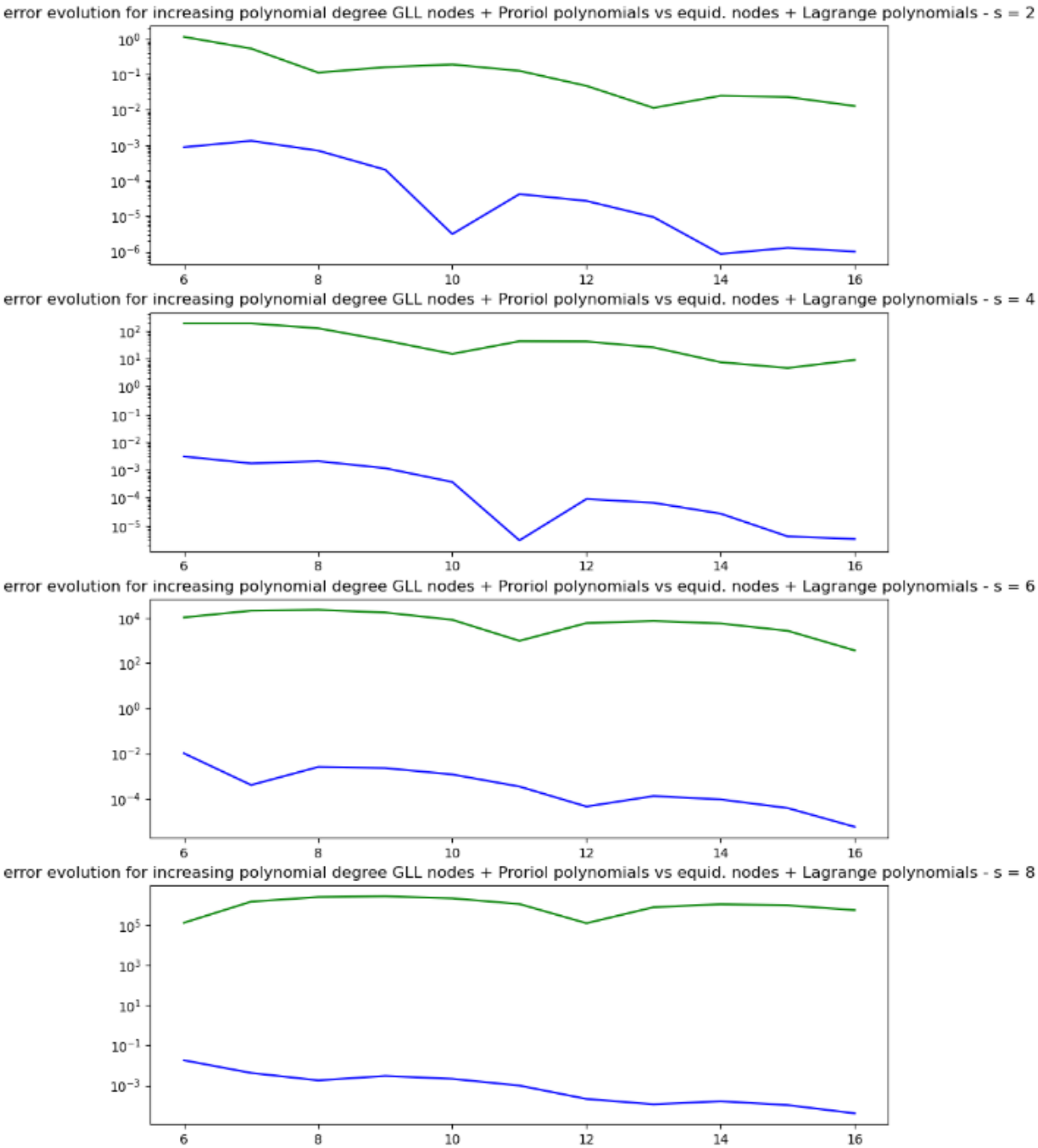
The plot for $f_{0.01,-1/3}$ is also similar to before, altough the error decreases more consistently, only starting to oscillate a bit from degree 20 on. The best approximation is also around 10 times better ($\approx 10^{-6}$ compared to before $\approx 10^{-5}$).
As before for $f_{2,1}$ we can again quantify how fast the exponential decay is by computing again the parameter α as for the equidistant interpolation, but this time taking it as average between the degrees 1 and 20. The result is plotted here below, we also plot a line $r(n) = Ce^{\alpha n}$ into the semilog plot to see whether the approximation for α is correct:



Maximal relative error evolution with GL Lagrange interpolation with $\epsilon = 0.01, s = -1/3$

We get an estimate for α of about -1.35, so the decay is a bit steeper than before in the equidistant case, where α was around -1.28.
On these two plots it seems that the new Gauss Lobatto Lagrange interpolation seems to perform only slightly better. However we want use the new interpolation on more singular functions, so we did a head to head performance test. Below are the evolutions of the maximal relative error for increasing polynomial degree for the functions $f_{0.1,s}$ with $s = 2, 4, 6, 8$. The green curve is the error evolution of the equidistant

Lagrange interpolation, the blue curve corresponds to the one of the Gauss Lobatto Lagrange interpolation:



Maximal relative error evolution with GL Lagrange interpolation with $\epsilon = 0.01, s = -1/3$

We see that not only the Gauss Lobatto interpolation is always at least 1000-times better, but for $s = 4, 6, 8$, while the equidistant interpolation fails to be an interpolation at all since the error is larger than 100%, the interpolation in the Gauss Lobatto nodes still gets errors of less than 10% even for not so high polynomial degrees, and it goes down to less than 1%!

5 Conclusion

The issue of diverging error on boundaries of the region that exists in one dimension, also arises again on the unit triangle. We were successfully able to work against this problem, analogously as we did in one dimension, by distributing nodes more densely at the edges and vertices of the unit triangle. While we couldnt solve entirely the issue of the instability for high polynomial degree interpolations, the interpolation allows to approximate far more singular functions than before. Its only downside is a higher computational cost, due to having to solve the Van der Monde system, even with using QR-decomposition. One further test for a performance comparison, is instead of always measuring the maximal relative error, is to use a mean ℓ^1 or mean ℓ^2 error, because the Gauss Lobatto interpolation has many small osciallatory errors at the edges and vertices, that could add up to possibly larger mean error. For example if we look at the error distributions for $f_{0.01,1}$ we note that the equidistant interpolation has one large error at the origin, but everywhere else the error is very small. The Gauss Lobatto has small errors everywhere at the boundaries of the unit triangle. It could be possible then that the mean errors increases faster for the Gauss Lobatto nodes, than for the equidistant nodes.

References

[1] M.G. Blyth and C.M. Porzidikis. *A Lobatto interpolation grid over the triangle.* IMA Journal of Applied Mathematics, 2005.

[2] John Burkardt. *The Finite Element Basis for Simplices in Arbitrary Dimensions.* Florida State University, 2013.

[3] Stefan A. Sauter. *Lecture script Numerical Methods for Elliptic and Parabolic Partial Differential Equationsr, HS2023.* University of Zurich, 2023.

[4] Stefan A. Sauter. *Lecture script Numerik I, FS2021.* University of Zurich, 2021.